Justin Figueroa

Dr. Raheja

Programming Languages

26 July 2026

Report Glossary

Comparison of Java and C++ Implementation (Key Evaluation Criteria)

The implementation of matrix operations in C++ and Java presented the opportunity to analyze the key evaluation criteria of both languages with respect to the limited scope of this domain, specifically matrix implementation. I believe the simplicity of the program has created a circumstance where C++ is slightly more readable than Java because of the effectivity of function prototyping, which is a boon to C++, and the nesting of methods inside of Java's public Matrix class, which may prove more useful in a complicated program that requires many classes, but in this case decreased Java's readability. With that being said, C++ prototyping also introduces the possibility of inconsistent functions if not implemented correctly which affects the writability of C++. Although this is one of my first Java programs, I found the language to be similar to C++ in terms of syntax and writability, but the language required an average of 150 less lines of code to implement the same matrix operations compared to the three C++ functions, therefore demonstrating its edge in writability. Java's reliability is demonstrated by its automatic garbage collection that could prevent memory leaks, dangling pointers, and other issues presented by the manual memory allocation used in some of C++'s implementation.

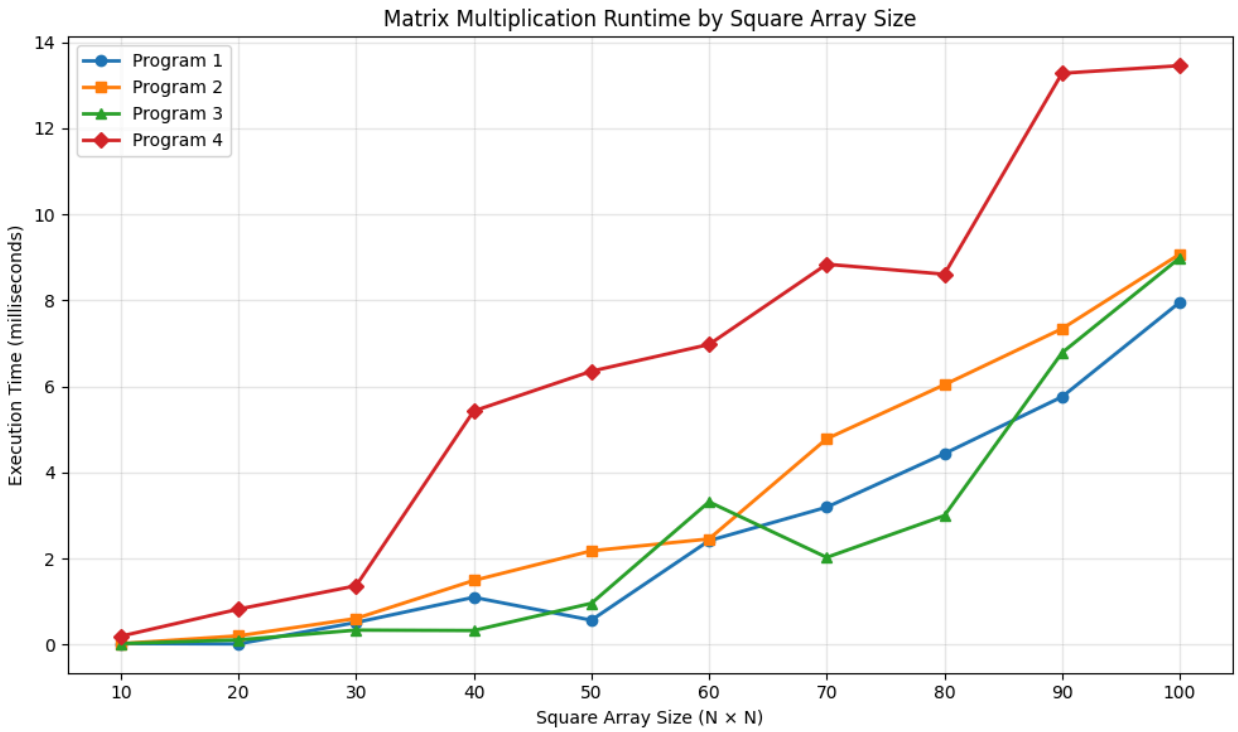
Time Comparison of Multiplication Function - Report

Methodology

In order to compare the runtime of the multiplication operation, I implemented a timer system corresponding to each language, which started before the multiplication function call and terminated immediately after. I excluded the displayMatrices function from all programs in order to emphasize the runtime of the operation only. Then, I programmed a random number generator that reads in the users desired square matrix dimensions and created a file that is compatible with the four programs file handler. I then generated the data points and ran the same values through each program. The time to complete is output to the terminal where I manually aggregated the data and uploaded it to desmos graphs. I used sizes 10, 20, 30, ..., 100 in order to demonstrate the differences in runtime for each program. The findings are as follows:

Results

Among the four implementations, program 3 demonstrated the fastest run time, showing a slower rate of growth as the array size increased. The program utilized operator overloading and pointer matrix allocation. It’s performance was closely matched by program 1, which used C++ standard 2d stack dynamic matrix operations. Program 2, on the other hand, showed a faster rate of growth as the matrix dimensions increased. Java's implementation rendered the slowest runtimes and grew significantly quicker than the other programs.



Program 1	Program 2	Program 3	Program 4
(10, .023)	(10, .020958)	(10,.021083)	(10,0.193042)
(20, .0143584)	(20, .20175)	(20, .108208)	(20, .822708)
(30, .512709)	(30, 0.60575ms)	(30, .335292)	(30, 1.365167)
(40, 1.09771)	(40, 1.48913)	(40, .325417)	(40, 5.428791)
(50, .566625)	(50, 2.178)	(50, .958041)	(50, 6.353709)
(60, 2.41429)	(60, 2.45504)	(60, 3.31596)	(60, 6.975958)
(70, 3.19271)	(70, 4.78292)	(70, 2.0255)	(70, 8.841292)
(80, 4.43675)	(80, 6.03908)	(80, 2.99604)	(80, 8.609083)
(90, 5.75821)	(90, 7.34112)	(90, 6.78379)	(90, 13.280125)
(100, 7.95529)	(100, 9.07525)	(100, 8.98617)	(100, 13.456542)

Conclusion

Although C++ features may sometimes make it less readable and reliable when compared to languages like Java, it clearly demonstrates its excellence as a quick programming language. Furthermore, features that may result in less readability and reliability if implemented well can create an overall more effective program. There is still a likely margin of error as a result of implementation of the various aspects of each program or the timer functionality. Admittedly, I don't often write in Java, and this is my first attempt at operator overloading, thanks to stackoverflow amongst other sources (hence the late submission). Also I would have liked to test much higher array dimensions to see how the programs held up with larger values. I will upload the source files and this report, but feel free to visit the Github repository at https://github.com/JustFigueroa/matrix-implementations-cpp-java.

```
justinfigueroa@13:07 programbuilds:./program1
Select entry method:
0.File entry
1.Manual entry
1
----------------------------------------
Enter the dimensions of your first matrix
----------------------------------------
matrix A rows: 2
matrix A columns: 2
----------------------------------------
Enter the dimensions of your second matrix
----------------------------------------
matrix B rows: 2
matrix B columns: 2
----------------------------------------
Enter values for Matrix A:
Matrix A [0][0]: 2
Matrix A [0][1]: 2
Matrix A [1][0]: 2
Matrix A [1][1]: 2
Enter values for Matrix B:
Matrix B [0][0]: 2
Matrix B [0][1]: 2
Matrix B [1][0]: 2
Matrix B [1][1]: 2
Matrix A:
2 2
2 2
Matrix B:
2 2
2 2
-----------------------------
Choose an operation to Perform
-----------------------------
0. Quit
1. Addition
2. Subtraction
3. Multiplication
3
*******
0.000167ms
```

```
justinfigueroa@13:09 programbuilds:./program2
Select entry method:
0.File entry
1.Manual entry
1
----------------------------------------
Enter the dimensions of your first matrix
----------------------------------------
matrix A rows: 2
matrix A columns: 2
----------------------------------------
Enter the dimensions of your second matrix
----------------------------------------
matrix B rows: 2
matrix B columns: 2
----------------------------------------
Enter values for Matrix A:
Matrix A [0][0]: 2
Matrix A [0][1]: 2
Matrix A [1][0]: 2
Matrix A [1][1]: 2
Enter values for Matrix B:
Matrix B [0][0]: 2
Matrix B [0][1]: 2
Matrix B [1][0]: 2
Matrix B [1][1]: 2
Matrix A:
2 2
2 2
Matrix B:
2 2
2 2
-----------------------------
Choose an operation to Perform
-----------------------------
0. Quit
1. Addition
2. Subtraction
3. Multiplication
3
*******
0.007666ms
```

```
justinfigueroa@13:10 programbuilds:./program3
----------------
1. Manual Entry
2. File Entry
Selection: 1
----------------------------------------
Enter the dimensions of your first matrix
----------------------------------------
matrix A rows: 2
matrix A columns: 2
----------------------------------------
Enter the dimensions of your second matrix
----------------------------------------
matrix B rows: 2
matrix B columns: 2
----------------------------------------
Enter values for Matrix A:
Matrix A [0][0]: 2
Matrix A [0][1]: 2
Matrix A [1][0]: 2
Matrix A [1][1]: 2
Enter values for Matrix B:
Matrix B [0][0]: 2
Matrix B [0][1]: 2
Matrix B [1][0]: 2
Matrix B [1][1]: 2
Matrix A:
2 2
2 2

Matrix B:
2 2
2 2

Choose an Operation:
0.New Matrix
1.Addition
2.Subtraction
3.Multiplication
3
*******
0.008583ms
```

```
justinfigueroa@13:11 sourcefiles:java Matrix
----------------------
Choose an entry method
----------------------
0.Quit program
1.Manual Entry
2.Read from file
Selection:
1
Matrix A rows: 2
Matrix A columns: 2
Matrix A[0] [0]: 2
Matrix A[0] [1]: 2
Matrix A[1] [0]: 2
Matrix A[1] [1]: 2
Matrix B rows: 2
Matrix B columns: 2
Matrix B[0] [0]: 2
Matrix B[0] [1]: 2
Matrix B[1] [0]: 2
Matrix B[1] [1]: 2
Matrix A:
2.0 2.0
2.0 2.0
Matrix B:
2.0 2.0
2.0 2.0
0. Select new matrices
1. Addition
2. Subtraction
3. Multiplication
Selection: 3
*****
0.021458ms*****
```

```
[justinfigueroa@13:04 programbuilds:ls
program1                program2                program3                RandomMatrixGenerator
[justinfigueroa@13:04 programbuilds:./RandomMatrixGenerator
This program will generate two random sqare arrays and export them to a file to be processes by matrix operation programs
Enter the square dimension of your arrays: 2
[justinfigueroa@13:04 programbuilds:ls
arrays.dat              program1                program2                program3                RandomMatrixGenerator
[justinfigueroa@13:04 programbuilds:cat arrays.dat
2 2
20.360936 6.244327
48.402956 8.506721
2 2
72.453296 22.539367
19.129850 15.393297
[justinfigueroa@13:04 programbuilds:./RandomMatrixGenerator
This program will generate two random sqare arrays and export them to a file to be processes by matrix operation programs
Enter the square dimension of your arrays: 4
[justinfigueroa@13:04 programbuilds:cat arrays.dat
4 4
23.758310 5.925520 90.209943 58.535564
7.221347 69.175178 27.248472 65.092182
4.297454 27.307117 50.736517 28.644636
30.418506 43.835810 48.480365 9.489561
4 4
91.052788 24.171573 51.614577 86.205387
53.958362 78.154808 47.832897 27.528867
77.664518 7.511316 42.695233 78.767943
52.810174 80.561858 3.156587 52.754867
justinfigueroa@13:04 programbuilds:
```